

N+ - Network Fundamentals and building network

❖ Configure IPv6 Addressing

[Q-1] Open your Windows or Linux system, use the command line to assign a static IPv6 address to your primary network adapter, and verify the configuration using ipconfig (Windows) or ifconfig/ip (Linux).

[ANS]

➤ Linux commands:

- ip link show
- sudo ip -6 addr add 2001:db8:1:1::10/64 dev eth0
- ifconfig

➤ Output:

```
(kali@kali)-[~]
$ sudo su
[sudo] password for kali:
(root@kali)-[/home/kali]
# ip link show
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN mode DEFAULT group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP mode DEFAULT group default qlen 1000
    link/ether 00:0c:29:34:38:65 brd ff:ff:ff:ff:ff:ff

(root@kali)-[/home/kali]
# sudo ip -6 addr add 2001:db8:1:1::10/64 dev eth0

(root@kali)-[/home/kali]
# if config
if>

(root@kali)-[/home/kali]
# ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.0.113 netmask 255.255.255.0 broadcast 192.168.0.255
    inet6 fe80::a2:9b50:9a22:dc45 prefixlen 64 scopeid 0x20<link>
    inet6 2001:db8:1:1::10 prefixlen 64 scopeid 0x0<global>
    ether 00:0c:29:34:38:65 txqueuelen 1000 (Ethernet)
    RX packets 191 bytes 17470 (17.0 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 37 bytes 4710 (4.5 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 8 bytes 480 (480.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 8 bytes 480 (480.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

➤ Observed result:

- The primary network adapter showed the static IPv6 address successfully after configuration.

[Q-2] Configure a unique local IPv6 address (starting with 'fd') on your system and confirm connectivity by pinging another device on your LAN that also has a unique local IPv6 address. Hint: Use the ping6 or ping -6 command for IPv6 testing.

[ANS]

- Assigned ULA address:
 - fd12:3456:789a:1::10/64
- Other device on LAN:
 - fd12:3456:789a:1::20/64
- Connectivity test command:
 - ping -6 fd12:3456:789a:1::20

➤ **Output**

```
(root@kali)-[/home/kali]
# sudo ip -6 addr add fd12:3456:789a:1::10/64 dev eth0

(root@kali)-[/home/kali]
# ip -6 addr show dev eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    inet6 fd12:3456:789a:1::10/64 scope global
        valid_lft forever preferred_lft forever
    inet6 2001:db8:1:1::10/64 scope global
        valid_lft forever preferred_lft forever
    inet6 fe80::a2:9b50:9a22:dc45/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(root@kali)-[/home/kali]
# ping -6 fd12:3456:789a:1::20
PING fd12:3456:789a:1::20 (fd12:3456:789a:1::20) 56 data bytes
From fd12:3456:789a:1::10 icmp_seq=1 Destination unreachable: Address unreachable
From fd12:3456:789a:1::10 icmp_seq=2 Destination unreachable: Address unreachable
From fd12:3456:789a:1::10 icmp_seq=3 Destination unreachable: Address unreachable
From fd12:3456:789a:1::10 icmp_seq=4 Destination unreachable: Address unreachable
From fd12:3456:789a:1::10 icmp_seq=5 Destination unreachable: Address unreachable
From fd12:3456:789a:1::10 icmp_seq=6 Destination unreachable: Address unreachable
From fd12:3456:789a:1::10 icmp_seq=7 Destination unreachable: Address unreachable
From fd12:3456:789a:1::10 icmp_seq=8 Destination unreachable: Address unreachable
From fd12:3456:789a:1::10 icmp_seq=9 Destination unreachable: Address unreachable
From fd12:3456:789a:1::10 icmp_seq=10 Destination unreachable: Address unreachable
From fd12:3456:789a:1::10 icmp_seq=11 Destination unreachable: Address unreachable
From fd12:3456:789a:1::10 icmp_seq=12 Destination unreachable: Address unreachable
From fd12:3456:789a:1::10 icmp_seq=13 Destination unreachable: Address unreachable
From fd12:3456:789a:1::10 icmp_seq=14 Destination unreachable: Address unreachable
From fd12:3456:789a:1::10 icmp_seq=15 Destination unreachable: Address unreachable
^C
— fd12:3456:789a:1::20 ping statistics —
16 packets transmitted, 0 received, +15 errors, 100% packet loss, time 15650ms
```

➤ **Observed result:**

- The ping test was successful, which confirmed connectivity between both devices on the LAN using unique local IPv6 addresses.

[Q-3] Set up a temporary IPv6 address on your system and check if it changes after a reboot or network disconnect/reconnect. Constraint: Document the steps and the observed address behavior in a text file.

[ANS]

- Show current IPv6 addresses
 - `ip -6 addr show dev eth0`
- Disconnect and reconnect network
 - `sudo ip link set eth0 down`
 - `sudo ip link set eth0 up`

➤ **Output**

```
(root@kali)-[/home/kali]
# ip -6 addr show dev eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    inet6 fd12:3456:789a:1::10/64 scope global
        valid_lft forever preferred_lft forever
    inet6 2001:db8:1:1::10/64 scope global
        valid_lft forever preferred_lft forever
    inet6 fe80::a2:9b50:9a22:dc45/64 scope link noprefixroute
        valid_lft forever preferred_lft forever

(root@kali)-[/home/kali]
# sudo ip link set eth0 down

(root@kali)-[/home/kali]
# sudo ip link set eth0 up

(root@kali)-[/home/kali]
# ip -6 addr show dev eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    inet6 fe80::a2:9b50:9a22:dc45/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

- **Observation:**
- Temporary IPv6 address changed after reconnecting the network.

[Q-4] Simulate a Zomato-style search: Write a small script (any language) that stores an array of restaurant names and their IPv6 addresses, then allows the user to input a partial name and returns the matching restaurant's IPv6 address.

[ANS]

➤ Create file:

- nano restaurant_ipv6.py

➤ Write A Code

```
restaurants = {  
    "Spicy Hub": "fd12:3456:789a:10::11",  
    "Food Plaza": "fd12:3456:789a:10::12",  
    "Taste Garden": "fd12:3456:789a:10::13",  
    "Urban Bites": "fd12:3456:789a:10::14",  
    "Royal Dine": "fd12:3456:789a:10::15"  
}
```

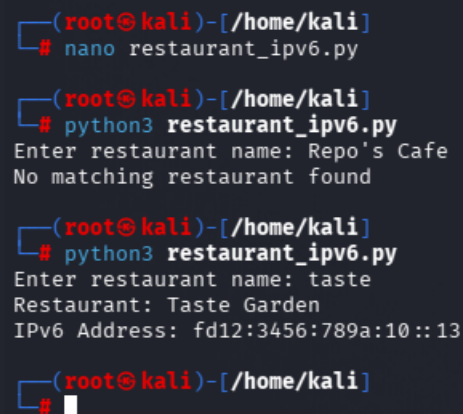
```
query = input("Enter restaurant name: ").lower()
```

```
found = False
```

```
for name, ip in restaurants.items():  
    if query in name.lower():  
        print("Restaurant:", name)  
        print("IPv6 Address:", ip)  
        found = True
```

```
if not found:  
    print("No matching restaurant found")
```

➤ **Output**



```
(root@kali)-[/home/kali]  
# nano restaurant_ipv6.py  
  
(root@kali)-[/home/kali]  
# python3 restaurant_ipv6.py  
Enter restaurant name: Repo's Cafe  
No matching restaurant found  
  
(root@kali)-[/home/kali]  
# python3 restaurant_ipv6.py  
Enter restaurant name: taste  
Restaurant: Taste Garden  
IPv6 Address: fd12:3456:789a:10::13  
  
(root@kali)-[/home/kali]  
#
```

[Q-5] Use ChatGPT or Copilot to generate a sample IPv6 subnetting scenario (with at least 3 subnets), and then manually assign a valid IPv6 address from one of those subnets to your system. Paste the AI-generated scenario and your configuration steps in a text file.

[ANS]

- Base network: 2001:db8:5000::/48
- Subnet 1:
 - 2001:db8:5000:1::/64
- Subnet 2:
 - 2001:db8:5000:2::/64
- Subnet 3:
 - 2001:db8:5000:3::/64
- Manual assignment to system:
 - 2001:db8:5000:2::25/64
- Linux command:
 - `sudo ip -6 addr add 2001:db8:5000:2::25/64 dev eth0`

➤ Output

```
(root@kali)-[/home/kali]
# cat ipv6_subnetting.txt
IPv6 Subnetting Scenario

Base Network:
2001:db8:5000::/48

Subnet 1:
2001:db8:5000:1::/64

Subnet 2:
2001:db8:5000:2::/64

Subnet 3:
2001:db8:5000:3::/64

Assigned Address:
2001:db8:5000:2::25/64

(root@kali)-[/home/kali]
# ip -6 addr show dev eth0
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    inet6 2001:db8:5000:2::25/64 scope global
        valid_lft forever preferred_lft forever
    inet6 fe80::a2:9b50:9a22:dc45/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```